PARTIE 2

Fonctionnement

d'une API REST

Cours de Bachelor 3 — Développement Back-end

Spring Boot 4.1.1 · Java 25

Durée	Jour	Prérequis
3 h	**Jour 1 — après-midi**	**Partie 1**

1. Introduction

Objectifs de cette partie

À la fin de cette partie, vous serez capables de :

- Expliquer ce que signifie REST et quels principes architecturaux le sous-tendent.
- Distinguer REST d'autres styles d'API (SOAP, GraphQL).
- Concevoir des URLs conformes aux conventions REST.
- Comprendre la notion de ressource, de représentation et d'état.
- Modéliser les endpoints d'une API REST sur papier avant de coder.
- Choisir les bons codes HTTP et les bonnes méthodes pour chaque opération.

Durée estimée

- Théorie : 1 h 30
- Pratique : 1 h 30

Prérequis

- Avoir suivi la Partie 1 (Qu'est-ce qu'une API distante ?).
- Savoir utiliser Postman ou curl pour envoyer des requêtes HTTP.

---

2. Concepts théoriques essentiels

2.1 Qu'est-ce que REST ?

REST signifie *Representational State Transfer*. Ce n'est pas un protocole ni un framework : c'est un **style architectural** défini par Roy Fielding dans sa thèse de doctorat en 2000.

REST définit un ensemble de **contraintes** que doit respecter une API pour être considérée comme « RESTful ». Ces contraintes favorisent la simplicité, la performance et la scalabilité.

En une phrase : REST, c'est l'art de manipuler des **ressources** via des **URLs** en utilisant les **méthodes HTTP** standard.

2.2 Les 6 contraintes de REST

1. Architecture client-serveur

Le client et le serveur sont séparés. Chacun peut évoluer indépendamment.



Le client ne connaît pas la base de données. Le serveur ne connaît pas l'interface graphique.

2. Sans état (Stateless)

Chaque requête contient **toutes les informations** nécessaires à son traitement. Le serveur ne conserve aucune information sur les requêtes précédentes.

Exemple concret :

```
❌ Mauvais — le serveur se souvient de l'utilisateur connecté :
GET /mes-commandes
→ Le serveur doit savoir "qui" appelle (état côté serveur)

☑ Bon — la requête est autonome :
GET /utilisateurs/42/commandes
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
→ Le serveur sait exactement quoi retourner grâce à l'URL et au token
```

Avantages du stateless :

- Le serveur peut être répliqué (load balancing) sans problème de synchronisation.
- Chaque requête est indépendante : plus facile à déboguer.
- Meilleure scalabilité.

3. Mise en cache (Cacheable)

Les réponses doivent indiquer si elles peuvent être mises en cache. Cela réduit le nombre de requêtes et améliore les performances.

```
HTTP/1.1 200 OK
Cache-Control: max-age=3600
ETag: "abc123"

{ "id": 1, "nom": "Dupont" }
```

Le client peut réutiliser cette réponse pendant 1 heure (3600 secondes) sans rappeler le serveur.

4. Interface uniforme

C'est la contrainte la plus importante. Elle se décline en quatre sous-principes :

- **Identification des ressources** : chaque ressource a une URL unique (`/utilisateurs/42`).
- **Manipulation via des représentations** : le client reçoit une représentation JSON de la ressource, pas la ressource elle-même.
- **Messages auto-descriptifs** : chaque requête/réponse contient assez d'informations pour être comprise (méthode, Content-Type, code de statut).
- **HATEOAS** : les réponses contiennent des liens vers les actions possibles (concept avancé, abordé en Partie 4).

5. Système en couches (Layered System)

Le client ne sait pas s'il communique directement avec le serveur final ou avec un intermédiaire (proxy, load balancer, CDN).

```
Client → CDN → Load Balancer → Serveur API → Base de données
```

Chaque couche ne voit que la couche immédiatement adjacente.

6. Code à la demande (optionnel)

Le serveur peut envoyer du code exécutable au client (JavaScript par exemple). Cette contrainte est rarement utilisée dans les API REST modernes.

2.3 La notion de ressource

En REST, tout est **ressource**. Une ressource est une entité conceptuelle identifiable par une URL.

Ressource	URL	Description
Un utilisateur	`/utilisateurs/42`	L'utilisateur ayant l'id 42
Tous les utilisateurs	`/utilisateurs`	La collection des utilisateurs
Les commandes d'un utilisateur	`/utilisateurs/42/commandes`	Les commandes de l'utilisateur 42
Une commande spécifique	`/utilisateurs/42/commandes/7`	La commande n°7 de l'utilisateur 42
Un produit	`/produits/15`	Le produit ayant l'id 15

Règle fondamentale : l'URL identifie la ressource, la méthode HTTP identifie l'action.

```
GET    /utilisateurs/42     → Lire l'utilisateur 42
PUT    /utilisateurs/42     → Remplacer l'utilisateur 42
DELETE /utilisateurs/42     → Supprimer l'utilisateur 42
```

La même URL, trois actions différentes. C'est l'essence de REST.

2.4 Convention de nommage des URLs

Les URLs REST suivent des conventions strictes. Un bon nommage rend l'API intuitive et prévisible.

Règles essentielles

Règle	Bon exemple	Mauvais exemple
Utiliser des **noms**, pas des verbes	`/utilisateurs`	`/getUtilisateurs`
Utiliser le **pluriel**	`/produits`	`/produit`
Utiliser des **minuscules**	`/utilisateurs`	`/Utilisateurs`
Séparer par des **tirets** (kebab-case)	`/categories-produits`	`/categoriesProduits`
Pas de slash final	`/utilisateurs`	`/utilisateurs/`
Pas d'extension de fichier	`/utilisateurs/42`	`/utilisateurs/42.json`

Ressources imbriquées

Quand une ressource appartient à une autre, on imbrique les URLs :

```
/utilisateurs                          → Tous les utilisateurs
/utilisateurs/42                       → L'utilisateur 42
/utilisateurs/42/commandes             → Les commandes de l'utilisateur 42
/utilisateurs/42/commandes/7           → La commande n°7 de l'utilisateur 42
/utilisateurs/42/commandes/7/articles  → Les articles de la commande 7
```

Conseil : ne dépassez pas 3 niveaux d'imbrication. Au-delà, l'URL devient difficile à lire et à maintenir.

Filtrage, tri et pagination

Pour les opérations de recherche, on utilise les **paramètres de requête** (query parameters) :

```
GET /produits?categorie=electronique          → Filtrer par catégorie
GET /produits?prix_min=10&prix_max=50         → Filtrer par fourchette de prix
GET /produits?tri=prix&ordre=asc              → Trier par prix croissant
GET /produits?page=2&taille=20                → Page 2, 20 éléments par page
```

2.5 Méthodes HTTP et opérations CRUD

REST s'appuie sur les méthodes HTTP pour exprimer les opérations. Le tableau ci-dessous montre la correspondance avec le CRUD (Create, Read, Update, Delete) :

Opération CRUD	Méthode HTTP	URL (exemple)	Corps	Réponse type
Create	`POST`	`/produits`	Oui (nouveau produit)	`201 Created` + produit créé
Read (tous)	`GET`	`/produits`	Non	`200 OK` + tableau de produits
Read (un)	`GET`	`/produits/15`	Non	`200 OK` + un produit
Update (total)	`PUT`	`/produits/15`	Oui (produit complet)	`200 OK` + produit mis à jour
Update (partiel)	`PATCH`	`/produits/15`	Oui (champs modifiés)	`200 OK` + produit mis à jour
Delete	`DELETE`	`/produits/15`	Non	`204 No Content`

PUT vs PATCH : quelle différence ?

PUT remplace la ressource **entièrement**. Il faut envoyer tous les champs.

```
PUT /produits/15
{
  "nom": "Clavier mécanique",
  "prix": 89.99,
  "categorie": "Informatique",
  "stock": 45
}
```

PATCH modifie **partiellement** la ressource. On envoie uniquement les champs à modifier.

```
PATCH /produits/15
{
  "prix": 79.99
}
```

Quand utiliser quoi ?

- `PUT` quand le client connaît et envoie la totalité de la ressource.
- `PATCH` quand on veut modifier un ou deux champs sans toucher au reste.

2.6 Les codes de statut HTTP en détail

Choisir le bon code de statut est une compétence essentielle. Voici un guide complet :

Succès (2xx)

Code	Nom	Quand l'utiliser
`200 OK`	Succès	GET, PUT, PATCH réussis
`201 Created`	Créé	POST réussi — une nouvelle ressource existe
`204 No Content`	Pas de contenu	DELETE réussi — rien à retourner
`304 Not Modified`	Non modifié	La ressource n'a pas changé (caching)

À propos du 304 — Optimisation du cache HTTP :

Le code `304 Not Modified` est utilisé pour optimiser les requêtes répétées. Au lieu de retransmettre une ressource entière, le serveur indique simplement que rien n'a changé.

```
Client (1ère requête)
    ↓
GET /api/produits/1
    ↓
Serveur
    ↓
HTTP 200 OK
ETag: "abc123"
{ "id": 1, "nom": "Clavier", "prix": 89.99 }

Client (2e requête, avec ETag)
    ↓
GET /api/produits/1
If-None-Match: "abc123"
    ↓
Serveur (l'ETag match → pas de changement)
    ↓
HTTP 304 Not Modified
(Pas de body — le client utilise sa copie en cache)
```

Avantage : économise la bande passante et améliore les performances pour les appels répétés.

Erreurs client (4xx)

Code	Nom	Quand l'utiliser
`400 Bad Request`	Requête invalide	JSON mal formé, champ manquant
`401 Unauthorized`	Non authentifié	Pas de token ou token invalide
`403 Forbidden`	Interdit	Authentifié mais pas autorisé
`404 Not Found`	Introuvable	La ressource n'existe pas
`405 Method Not Allowed`	Méthode non permise	DELETE sur une ressource en lecture seule
`409 Conflict`	Conflit	Email déjà utilisé, doublon
`422 Unprocessable Entity`	Entité non traitable	Validation échouée (email invalide, etc.)

Erreurs serveur (5xx)

Code	Nom	Quand l'utiliser
`500 Internal Server Error`	Erreur interne	Bug non géré côté serveur
`503 Service Unavailable`	Service indisponible	Maintenance, surcharge

Erreur fréquente des débutants : retourner `200 OK` avec un message d'erreur dans le corps. C'est une mauvaise pratique. Le code de statut doit refléter le résultat réel de l'opération.

```
❌ Mauvais :
HTTP 200 OK
{ "erreur": "Utilisateur non trouvé" }

☑ Bon :
HTTP 404 Not Found
{ "message": "Utilisateur non trouvé", "code": "USER_NOT_FOUND" }
```

2.7 Structure d'une réponse JSON bien conçue

Le but est d'assurer la **cohérence** et la **prévisibilité** des réponses. Un client qui consomme votre API doit savoir exactement où trouver les données, peu importe l'endpoint appelé. Une structure uniforme réduit les erreurs d'intégration et simplifie la maintenance.

Règles générales de nommage

Avant de voir les structures, quelques conventions importantes :

Règle	Bon exemple	Mauvais exemple
Utiliser le **camelCase** pour les champs	`dateCreation`, `totalElements`	`date_creation`, `TotalElements`
Préférer des **noms explicites**	`dateCreation`	`dc`, `date1`
Utiliser **ISO 8601** pour les dates	`"2025-09-15T10:30:00Z"`	`"15/09/2025"`, `1726391400`
Ne jamais retourner `null` sans raison	Omettre le champ s'il est absent	`"prenom": null`
Champs booléens au format positif	`"actif": true`	`"nonActif": false`

Réponse pour une ressource unique

Retournez directement l'objet — sans enveloppe superflue. Simple, direct, lisible.

```
{
  "id": 42,
  "nom": "Dupont",
  "prenom": "Marie",
  "email": "marie.dupont@email.com",
  "dateCreation": "2025-09-15T10:30:00Z",
  "actif": true
}
```

À éviter — enveloppe inutile :

```
{
  "success": true,
  "data": {
    "id": 42,
    "nom": "Dupont"
  },
  "error": null
}
```

> Envelopper chaque réponse dans `data` n'apporte rien quand le code HTTP exprime déjà le résultat. Le code `200 OK` signifie déjà « succès ».

Réponse pour une collection (avec pagination)

Quand une collection peut contenir des centaines ou milliers d'éléments, on ne renvoie **jamais tout d'un coup**. La pagination est obligatoire.

```
{
  "contenu": [
    { "id": 1, "nom": "Dupont", "prenom": "Marie" },
    { "id": 2, "nom": "Martin", "prenom": "Paul" },
    { "id": 3, "nom": "Bernard", "prenom": "Sophie" }
  ],
  "page": 0,
  "taille": 20,
  "totalElements": 150,
  "totalPages": 8,
  "premierePage": true,
  "dernierePage": false
}
```

Explication des champs de pagination :

Champ	Type	Description
`contenu`	tableau	Les éléments de la page courante
`page`	entier	Numéro de la page (commence à 0)
`taille`	entier	Nombre d'éléments par page
`totalElements`	entier	Nombre total d'éléments dans la base
`totalPages`	entier	Nombre total de pages
`premierePage`	booléen	Vrai si c'est la première page
`dernierePage`	booléen	Vrai si c'est la dernière page

Comment appeler une page spécifique :

```
GET /api/v1/utilisateurs?page=2&taille=20
```

> Spring Boot avec Spring Data expose automatiquement cette structure via `Page<T>`. Vous n'avez pas à la construire manuellement.

Réponse d'erreur

Une erreur bien structurée aide le client à **comprendre ce qui s'est passé** et à **afficher un message utile** à l'utilisateur.

```
{
  "timestamp": "2025-09-15T10:30:00Z",
  "statut": 404,
  "erreur": "Not Found",
  "message": "Aucun produit trouvé avec l'id 999",
  "chemin": "/api/produits/999"
}
```

Pour les erreurs de validation (400 / 422), listez chaque champ en erreur :

```
{
  "timestamp": "2025-09-15T10:35:00Z",
  "statut": 422,
  "erreur": "Unprocessable Entity",
  "message": "Validation échouée — 2 erreur(s) détectée(s)",
  "chemin": "/api/v1/utilisateurs",
  "erreurs": [
    { "champ": "email",     "valeur": "dupont@",      "probleme": "Format d'email
invalide" },
    { "champ": "age",       "valeur": "15",            "probleme": "L'âge doit être
supérieur ou égal à 18" }
  ]
}
```

Comparaison — bonnes et mauvaises pratiques

```
❌ Mauvais — retourner 200 OK avec un message d'erreur dans le corps
HTTP 200 OK
{ "erreur": "Utilisateur non trouvé" }

❌ Mauvais — retourner une erreur sans contexte
HTTP 404 Not Found
{ "message": "Not found" }

☑ Bon — code HTTP cohérent + corps informatif
HTTP 404 Not Found
{
  "timestamp": "2025-09-15T10:30:00Z",
  "statut": 404,
  "erreur": "Not Found",
  "message": "Aucun utilisateur trouvé avec l'id 42",
  "chemin": "/api/v1/utilisateurs/42"
}
```

Récapitulatif — quelle structure pour quelle situation ?

Situation	Structure attendue	Code HTTP
Lecture d'une ressource	Objet JSON direct	`200 OK`
Création d'une ressource	Objet JSON créé	`201 Created`
Liste / collection	Objet avec `contenu` + pagination	`200 OK`
Suppression	Pas de corps	`204 No Content`
Ressource introuvable	Objet d'erreur structuré	`404 Not Found`
Validation échouée	Objet d'erreur + liste des champs	`422 Unprocessable Entity`
Erreur serveur	Objet d'erreur (sans détails internes)	`500 Internal Server Error`

2.8 REST vs SOAP vs GraphQL

Critère	REST	SOAP	GraphQL
Style	Architectural	Protocole	Langage de requête
Format	JSON (principalement)	XML uniquement	JSON
Transport	HTTP	HTTP, SMTP, etc.	HTTP
Flexibilité	Moyenne	Faible	Élevée
Complexité	Simple	Complexe	Moyenne
Cas d'usage	API web, microservices	Services entreprise, banques	Applications mobiles, dashboards
Sur-requêtage	Possible (overfetching)	Fréquent	Résolu par design

Pourquoi REST domine aujourd'hui ?

- Simplicité : il s'appuie sur HTTP, un protocole que tout le monde connaît.
- Léger : JSON est beaucoup moins verbeux que XML.
- Universel : tout langage capable d'envoyer des requêtes HTTP peut consommer une API REST.
- Outillage : Postman, Swagger, curl — l'écosystème est immense.

2.9 Le versionnement d'une API

Quand une API évolue, il faut protéger les clients existants. Le versionnement permet de faire coexister plusieurs versions.

Stratégie n°1 : dans l'URL (la plus courante)

```
GET /api/v1/utilisateurs
GET /api/v2/utilisateurs
```

Stratégie n°2 : dans l'en-tête

```
GET /api/utilisateurs
Accept: application/vnd.monapi.v2+json
```

Stratégie n°3 : par paramètre de requête

```
GET /api/utilisateurs?version=2
```

Recommandation pour ce cours : utilisez le versionnement dans l'URL (`/api/v1/...`). C'est la méthode la plus lisible et la plus répandue.

> **Nouveauté Spring Framework 7 (Spring Boot 4) : le versionnement intégré.** Spring MVC sait désormais choisir la méthode du contrôleur selon la version demandée par le client, par exemple dans un en-tête HTTP :

```
spring.mvc.apiversion.use.header=X-Version
spring.mvc.apiversion.default=1.0


@RestController
@RequestMapping("/api/utilisateurs")
public class UtilisateurController {

    @GetMapping(version = "1.0")
    public List<UtilisateurV1> listerV1() { ... }

    @GetMapping(version = "2.0+") // version 2.0 et suivantes
    public List<UtilisateurV2> listerV2() { ... }
}
```

> Une version inconnue est rejetée avec une erreur `400`. Dans ce cours, nous conservons le préfixe `/api/v1/` dans l'URL, plus simple à tester avec un navigateur ou `curl`.

---

3. Mise en pratique guidée

3.1 Atelier de conception : l'API d'une librairie en ligne

Cet atelier se fait **sur papier** (ou sur un tableau blanc). Le but est de concevoir les endpoints d'une API REST avant d'écrire la moindre ligne de code.

Contexte

Vous devez concevoir l'API d'une librairie en ligne qui gère des **livres** et des **auteurs**. Un livre a un seul auteur. Un auteur peut avoir plusieurs livres.

Étape 1 : identifier les ressources

Listez les entités principales et leurs attributs :

- **Livre** : titre, isbn, prix, année de publication, auteur
- **Auteur** : nom, prénom, nationalité

Étape 2 : définir les URLs

Complétez le tableau suivant en choisissant la méthode HTTP, l'URL et le code de réponse appropriés :

Action	Méthode	URL	Corps	Réponse
Lister tous les livres				
Obtenir un livre				
Créer un livre				

Action	Méthode	URL	Corps	Réponse
Modifier un livre (complet)				
Modifier un livre (partiel)				
Supprimer un livre				
Lister tous les auteurs				
Obtenir un auteur				
Livres d'un auteur				
Rechercher des livres par titre				

Étape 3 : définir les modèles JSON

Rédigez la structure JSON attendue en réponse pour :

- Une ressource **Livre** (avec son auteur imbriqué)
- Une ressource **Auteur**
- Une **collection paginée** de livres
- Le **corps de création** envoyé par le client (POST)

Étape 4 : anticiper les erreurs

Pour chaque situation, précisez le code HTTP et rédigez un corps de réponse JSON structuré :

Situation	Code HTTP	Corps JSON attendu
Livre introuvable		
ISBN déjà existant		
Champ titre manquant		
Auteur référencé inexistant		

> **Corrigé :** `Corrigés/02-Corrige-Atelier-API-Librairie.md`

3.2 Tester la conception avec Postman

Maintenant, vérifions votre compréhension en simulant des appels API sur JSONPlaceholder. L'objectif est de faire le lien entre la conception théorique et les vrais appels HTTP.

Scénario 1 : Lire une collection

```
curl -s https://jsonplaceholder.typicode.com/posts | python3 -m json.tool | more
```

Observez la structure : c'est un tableau JSON `[...]` contenant des objets.

Scénario 2 : Lire une ressource spécifique

```
curl -s https://jsonplaceholder.typicode.com/posts/1 | python3 -m json.tool
```

Observez : c'est un objet `{...}` unique.

Scénario 3 : Filtrer une collection

```
curl -s "https://jsonplaceholder.typicode.com/posts?userId=1" | python3 -m json.tool
```

Observez : seuls les posts de l'utilisateur 1 sont retournés. Le filtre passe en query parameter.

Scénario 4 : Ressources imbriquées

```
curl -s https://jsonplaceholder.typicode.com/posts/1/comments | python3 -m json.tool
```

Observez : les commentaires **du post 1**. L'imbrication `/posts/1/comments` reflète la relation de possession.

3.3 Analyser les en-têtes HTTP

Dans Postman, après un appel GET, cliquez sur l'onglet **Headers** de la réponse. Identifiez :

- `Content-Type` → devrait être `application/json; charset=utf-8`
- `Cache-Control` → politique de cache
- `X-Powered-By` → technologie du serveur (information souvent masquée en production)

Avec curl, utilisez l'option `-i` pour afficher les en-têtes :

```
curl -i https://jsonplaceholder.typicode.com/posts/1
```

3.4 Exercice Java : construire un mini-framework de requêtes REST

Créons une classe utilitaire réutilisable pour simplifier les appels REST en Java.

> Nous l'appelons `ClientRestSimple` pour ne pas la confondre avec `RestClient`, le client HTTP fourni par Spring, que vous utiliserez dans le Document 6.

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

/**
 * Classe utilitaire pour effectuer des appels REST.
 * Encapsule les détails de HttpClient pour simplifier l'usage.
 */
public class ClientRestSimple {

    private final HttpClient client;
```

```
    private final String baseUrl;

    /**
     * Crée un ClientRestSimple pointant vers une URL de base.
     * @param baseUrl L'URL racine de l'API (ex :
"https://jsonplaceholder.typicode.com")
     */
    public ClientRestSimple(String baseUrl) {
        this.client = HttpClient.newHttpClient();
        this.baseUrl = baseUrl;
    }

    /**
     * Effectue une requête GET.
     * @param chemin Le chemin relatif (ex : "/posts/1")
     * @return La réponse HTTP
     */
    public HttpResponse<String> get(String chemin) throws Exception {
        HttpRequest requete = HttpRequest.newBuilder()
                .uri(URI.create(baseUrl + chemin))
                .header("Accept", "application/json")
                .GET()
                .build();

        return client.send(requete, HttpResponse.BodyHandlers.ofString());
    }

    /**
     * Effectue une requête POST avec un corps JSON.
     * @param chemin Le chemin relatif (ex : "/posts")
     * @param corpsJson Le corps de la requête en JSON
     * @return La réponse HTTP
     */
    public HttpResponse<String> post(String chemin, String corpsJson) throws
Exception {
        HttpRequest requete = HttpRequest.newBuilder()
                .uri(URI.create(baseUrl + chemin))
                .header("Content-Type", "application/json")
                .header("Accept", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(corpsJson))
                .build();

        return client.send(requete, HttpResponse.BodyHandlers.ofString());
    }

    /**
     * Effectue une requête PUT avec un corps JSON.
     * @param chemin Le chemin relatif (ex : "/posts/1")
     * @param corpsJson Le corps de la requête en JSON
     * @return La réponse HTTP
     */
    public HttpResponse<String> put(String chemin, String corpsJson) throws Exception
{
        HttpRequest requete = HttpRequest.newBuilder()
                .uri(URI.create(baseUrl + chemin))
                .header("Content-Type", "application/json")
                .header("Accept", "application/json")
                .PUT(HttpRequest.BodyPublishers.ofString(corpsJson))
                .build();

        return client.send(requete, HttpResponse.BodyHandlers.ofString());
    }

    /**
     * Effectue une requête DELETE.
     * @param chemin Le chemin relatif (ex : "/posts/1")
     * @return La réponse HTTP
     */
    public HttpResponse<String> delete(String chemin) throws Exception {
```

```
        HttpRequest requete = HttpRequest.newBuilder()
                .uri(URI.create(baseUrl + chemin))
                .DELETE()
                .build();

        return client.send(requete, HttpResponse.BodyHandlers.ofString());
    }

    /**
     * Affiche les détails d'une réponse HTTP de manière formatée.
     */
    public static void afficherReponse(String operation, String methode,
                                       String chemin, HttpResponse<String> reponse)
{
        System.out.println("=== " + operation + " ===");
        System.out.println(methode + " " + chemin + " → " + reponse.statusCode());
        System.out.println("Réponse : " + reponse.body());
        System.out.println();
    }
}
```

Programme principal utilisant ce client :

```
public class DemoClientRestSimple {

    public static void main(String[] args) throws Exception {

        ClientRestSimple api = new
ClientRestSimple("https://jsonplaceholder.typicode.com");

        // 1. GET — Lire un post
        var reponseGet = api.get("/posts/1");
        ClientRestSimple.afficherReponse("LIRE", "GET", "/posts/1", reponseGet);

        // 2. POST — Créer un post
        String nouveauPost = """
                {
                    "title": "Apprendre REST",
                    "body": "REST est un style architectural simple et puissant.",
                    "userId": 1
                }
                """;
        var reponsePost = api.post("/posts", nouveauPost);
        ClientRestSimple.afficherReponse("CREER", "POST", "/posts", reponsePost);

        // 3. PUT — Modifier un post
        String postModifie = """
                {
                    "id": 1,
                    "title": "Titre modifié",
                    "body": "Contenu mis à jour.",
                    "userId": 1
                }
                """;
        var reponsePut = api.put("/posts/1", postModifie);
        ClientRestSimple.afficherReponse("MODIFIER", "PUT", "/posts/1", reponsePut);

        // 4. DELETE — Supprimer un post
        var reponseDelete = api.delete("/posts/1");
        ClientRestSimple.afficherReponse("SUPPRIMER", "DELETE", "/posts/1",
reponseDelete);
    }
}
```

Ce que ce code enseigne :

- L'encapsulation des appels HTTP dans une classe utilitaire.

- La correspondance entre les méthodes CRUD et les méthodes HTTP.
- L'importance du `Content-Type` et de l'en-tête `Accept`.
- L'utilisation des Text Blocks Java pour écrire du JSON lisible.

---

4. Exercices pratiques

Exercice 1 — Conception d'API sur papier (Facile)

Objectif : Maîtriser les conventions de nommage REST.

Consigne : Vous devez concevoir l'API REST d'une application de gestion de **tâches** (to-do list). L'application gère des **projets** et des **tâches**. Chaque projet contient plusieurs tâches. Chaque tâche appartient à un seul projet.

Modèle de données :

- **Projet** : id, nom, description, dateCreation
- **Tâche** : id, titre, description, statut (A_FAIRE, EN_COURS, TERMINEE), priorite (BASSE, MOYENNE, HAUTE), projetId

Travail demandé :

1. Listez tous les endpoints nécessaires (au moins 10) en précisant : la méthode HTTP, l'URL, le corps éventuel et le code de réponse attendu.
2. Écrivez un exemple de requête et de réponse JSON pour chaque endpoint.
3. Identifiez au moins 4 scénarios d'erreur possibles avec leurs codes HTTP.

Travail en binôme : Un étudiant conçoit les endpoints des projets, l'autre ceux des tâches. Fusionnez et vérifiez la cohérence ensemble.

Résultat attendu : Un tableau complet des endpoints et des exemples JSON sur papier ou dans un fichier texte.

Exercice 2 — Identifier les erreurs REST (Intermédiaire)

Objectif : Repérer les violations des conventions REST.

Consigne : L'API suivante a été mal conçue. Pour chaque endpoint, identifiez l'erreur et proposez la correction.

#	Méthode	URL	Problème ?
1	`GET`	`/getUtilisateurs`	?
2	`POST`	`/utilisateur/creer`	?
3	`GET`	`/Produit/42`	?
4	`DELETE`	`/utilisateurs/42/`	?
5	`PUT`	`/utilisateurs/modifier/42`	?
6	`GET`	`/utilisateurs/42.json`	?
7	`POST`	`/utilisateurs/42`	?
8	`GET`	`/getCommandesParUtilisateur?id=42`	?

#	Méthode	URL	Problème ?
9	`DELETE`	`/supprimerTout`	?
10	`PATCH`	`/utilisateur/42/updateEmail`	?

Réponses attendues (format) :

```
1. GET /getUtilisateurs
   Erreur : verbe dans l'URL
   Correction : GET /utilisateurs

2. POST /utilisateur/creer
   Erreur : verbe dans l'URL + singulier
   Correction : POST /utilisateurs

...
```

Exercice 3 — Client REST avancé en Java (Avancé)

Objectif : Créer un programme Java interactif qui consomme une API REST.

Consigne : En réutilisant la classe `ClientRestSimple` vue en cours, créez un programme `GestionnairePostsApi` qui propose un menu interactif dans le terminal :

```
=== Gestionnaire de Posts ===
1. Lister tous les posts (limité aux 5 premiers)
2. Afficher un post par son ID
3. Créer un nouveau post
4. Modifier un post existant
5. Supprimer un post
6. Rechercher les posts d'un utilisateur
0. Quitter
Votre choix :
```

Exigences :

- Utilisez `java.util.Scanner` pour la saisie utilisateur.
- Pour chaque opération, affichez la méthode HTTP, l'URL et le code de réponse.
- L'option 3 (Créer) doit demander le titre, le corps et l'userId à l'utilisateur.
- L'option 6 (Rechercher) doit utiliser un query parameter : `/posts?userId=X`.
- Gérez les erreurs (ID invalide, connexion impossible).

Squelette de départ :

```
import java.util.Scanner;

public class GestionnairePostsApi {

    private static final ClientRestSimple api =
            new ClientRestSimple("https://jsonplaceholder.typicode.com");

    public static void main(String[] args) throws Exception {
        Scanner scanner = new Scanner(System.in);
        boolean continuer = true;

        while (continuer) {
            afficherMenu();
            int choix = scanner.nextInt();
```

```
            scanner.nextLine(); // consommer le retour à la ligne

            switch (choix) {
                case 1 -> listerPosts();
                case 2 -> {
                    System.out.print("ID du post : ");
                    int id = scanner.nextInt();
                    afficherPost(id);
                }
                case 3 -> creerPost(scanner);
                case 4 -> modifierPost(scanner);
                case 5 -> {
                    System.out.print("ID du post à supprimer : ");
                    int id = scanner.nextInt();
                    supprimerPost(id);
                }
                case 6 -> {
                    System.out.print("ID de l'utilisateur : ");
                    int userId = scanner.nextInt();
                    rechercherParUtilisateur(userId);
                }
                case 0 -> continuer = false;
                default -> System.out.println("Choix invalide.");
            }
        }

        System.out.println("Au revoir !");
    }

    private static void afficherMenu() {
        System.out.println("\n=== Gestionnaire de Posts ===");
        System.out.println("1. Lister tous les posts (5 premiers)");
        System.out.println("2. Afficher un post par ID");
        System.out.println("3. Créer un nouveau post");
        System.out.println("4. Modifier un post");
        System.out.println("5. Supprimer un post");
        System.out.println("6. Rechercher par utilisateur");
        System.out.println("0. Quitter");
        System.out.print("Votre choix : ");
    }

    // TODO : implémenter chaque méthode en utilisant ClientRestSimple
    private static void listerPosts() throws Exception {
        // Appeler api.get("/posts") et afficher les 5 premiers
    }

    private static void afficherPost(int id) throws Exception {
        // Appeler api.get("/posts/" + id) et gérer le 404
    }

    private static void creerPost(Scanner scanner) throws Exception {
        // Demander titre, body, userId puis appeler api.post(...)
    }

    private static void modifierPost(Scanner scanner) throws Exception {
        // Demander l'id, les nouvelles valeurs, puis appeler api.put(...)
    }

    private static void supprimerPost(int id) throws Exception {
        // Appeler api.delete("/posts/" + id)
    }

    private static void rechercherParUtilisateur(int userId) throws Exception {
        // Appeler api.get("/posts?userId=" + userId)
    }
}
```

Travail en binôme : Un étudiant implémente les opérations de lecture (options 1, 2, 6), l'autre implémente les opérations d'écriture (options 3, 4, 5). Intégrez et testez ensemble.

Bonus : Ajoutez une option 7 qui affiche les commentaires d'un post (endpoint `/posts/{id}/comments`).

---

5. Résumé des points clés

Ce qu'il faut retenir :

- **REST** est un style architectural basé sur la manipulation de **ressources** via des URLs et des méthodes HTTP.
- Les 6 contraintes de REST sont : client-serveur, sans état, cache, interface uniforme, système en couches et code à la demande.
- Une URL REST identifie une **ressource** (nom, pluriel, minuscules, pas de verbe).
- Les **méthodes HTTP** expriment l'action : GET (lire), POST (créer), PUT (remplacer), PATCH (modifier partiellement), DELETE (supprimer).
- Les **codes de statut** doivent refléter fidèlement le résultat : 2xx pour le succès, 4xx pour les erreurs client, 5xx pour les erreurs serveur.
- Le **versionnement** (`/api/v1/`) protège les clients existants quand l'API évolue.
- On conçoit les endpoints **sur papier** avant de coder.

Compétences acquises :

Compétence	Vérifié par
Définir REST et ses contraintes	Théorie + discussion
Concevoir des URLs REST	Exercice 1 (sur papier)
Détecter les erreurs de conception	Exercice 2 (identification)
Consommer une API REST en Java	Exercice 3 (client interactif)
Choisir les bons codes de statut	Tous les exercices

---

6. Pour aller plus loin

> Les pistes d'approfondissement de cette partie sont regroupées dans le document `Aller-Plus-Loin/02-Aller-Plus-Loin-Fonctionnement-API-REST.md` :
>
> - La thèse de Roy Fielding (chapitre 5) : l'intention originale de REST
> - La spécification OpenAPI
> - L'API GitHub, exemple d'API REST bien conçue
> - HATEOAS et le modèle de maturité de Richardson
> - Le versionnement d'API intégré à Spring Framework 7

Prochaine partie → Mise en œuvre d'une API REST avec Java et Spring Boot : créer votre premier projet, vos premiers contrôleurs et votre première base de données.